BRANCH COVERAGE REPORT

Electronic Store Inventory Management System

Academic Software Engineering Testing Deliverable | Decision Branch & Conditional Path Analysis

1. Introduction

Branch coverage (also termed decision coverage) is an advanced structural testing metric in software engineering that measures whether each decision point in an application's source code has been evaluated under all possible outcomes. Whereas statement coverage merely requires that a conditional statement line be visited, branch coverage requires that both the **True** and **False** evaluation paths of every decision construct (such as if, elif, else, while, for, and boolean short-circuit expressions) be traversed during automated test execution.

Within the Electronic Store Inventory Management System, complex conditional logic governs business-critical operations including inventory reorder thresholds, multi-tier user role authorization, purchase order approval workflows, sales transaction validation, customer loyalty discount tiers, and partial product receipt calculations. This report provides an empirical analysis of decision branch execution across the entire production codebase, establishing the depth of conditional path validation achieved by the 373-test automated suite.

2. Objectives of Branch Coverage Measurement

- **Evaluate Decision Logic Execution:** Measure the exact proportion of conditional decision branches traversed across both True and False outcomes during automated test runs.
- **Identify Unexercised Decision Paths:** Detect partial branches where a condition was evaluated in only one direction (e.g., an if condition was satisfied, but the implicit or explicit else path was never exercised).
- **Verify Defensive Conditional Guards:** Validate that boundary checks, null guards, status state machine rules, and validation rejections execute correctly under alternative inputs.
- **Expose Hidden Logic Hazards:** Uncover conditional fallbacks, unintended fallthroughs, and dead branches that cannot be revealed through simple statement coverage alone.
- **Establish Rigorous Verification:** Confirm that mission-critical business transactions (stock updates, sales deductions, order receipts, and financial calculations) operate correctly under both valid and invalid input branches.

3. Testing and Coverage Environment

Branch coverage was measured directly within the local repository virtual environment using pytest-cov and coverage.py with branch measurement explicitly enabled. A fresh test run ensured that branch counters reflected the current state of the test suite without cached coverage data.

Component / Tool	Specification / Version
Programming Language	Python 3.14.6 (64-bit)
Web Application Framework	Flask 3.1.0
Test Execution Engine	Pytest 9.1.1
Branch Instrumentation Tool	coverage.py 7.17.2 via pytest-cov 7.1.0 (--cov-branch)
Database Engine	SQLite 3 (In-Memory & Ephemeral File Fixtures)
Object-Relational Mapping (ORM)	SQLAlchemy 2.0 / Flask-SQLAlchemy 3.1
Operating System	macOS and Windows (both were used)
Virtual Environment Path	./venv (Repository Virtualenv)
Measurement Invocation	pytest --cov=app --cov=scripts --cov-branch --cov-report=term-missing

4. Branch Coverage Measurement Methodology and Scope

Branch coverage was captured by configuring coverage.py with its Abstract Syntax Tree (AST) parser to insert tracking probes at every conditional branch in the source code. For every if statement, the tool identifies two distinct branch transitions: one leading into the body of the if block (the True outcome) and one bypassing the block to the next statement or else clause (the False outcome).

The measured production scope is strictly limited to app/ and scripts/, comprising exactly **28 Python source files**. All virtual environment dependencies, third-party libraries, and the test suite itself were strictly excluded. A total of 16 modules contain executable decision branches, while 12 modules contain pure declarative model structures or package initializers with zero conditional branch points.

5. Overall Branch Coverage Results

The empirical branch coverage results compiled from the fresh execution of the complete automated test suite are detailed below:

Branch Coverage Metric	Empirical Value	Engineering Interpretation
Total Decision Branches Identified	694	Total decision-outcome transitions across 28 source files
Fully Covered Branches	635	Decision transitions evaluated in both True and False directions
Missing / Partial Branches	59	Decision transitions evaluated in only one direction or unreached
Pure Branch Coverage (Exact)	**91.50%**	635 / 694 = 91.49856%
Pure Branch Coverage (Display)	**91%**	Rounded display integer per coverage.py standard
Combined Statement + Branch Coverage	96.10% (Display: 96%)	(1,950 stmts + 635 branches) / (1,996 + 694) = 96.10%
Automated Tests Executed	373 passed / 0 failed	100% test pass rate across 20 test modules

6. Module-Wise Branch Coverage Breakdown

The following table provides an exhaustive account of branch coverage across all 28 measured source modules. Modules containing zero decision branches are explicitly identified as N/A (0 branches) rather than misleadingly presented as 100%. Missing branch transitions indicate the source line of the condition and the unreached destination.

Source File Path	Total Branches	Covered	Miss	Branch Cov %	Missing Branch Transitions
app/__init__.py	12	12	0	100.00%	None (100% covered)
app/config.py	0	0	0	N/A	N/A (0 branches)
app/business/__init__.py	0	0	0	N/A	N/A (0 branches)
app/business/audit.py	0	0	0	N/A	N/A (0 branches)
app/business/password_reset_email.py	4	4	0	100.00%	None (100% covered)
app/business/purchase_order.py	142	122	20	85.92%	27->28, 42->43, 145->146, 167->174, 193->194, 205->206, 210->211, 219->220, 225->237, 241->244, 269->270, 281->282, 286->287, 304->305, 336->337, 358->359, 372->373, 386->387, 403->406, 438->439
app/business/report_exports.py	12	12	0	100.00%	None (100% covered)
app/business/sales.py	26	24	2	92.31%	69->70, 123->124
app/business/security.py	10	10	0	100.00%	None (100% covered)
app/business/stock.py	58	55	3	94.83%	20->21, 159->164, 176->193
app/data/__init__.py	0	0	0	N/A	N/A (0 branches)

Source File Path	Total Branches	Covered	Miss	Branch Cov %	Missing Branch Transitions
app/data/extensions.py	0	0	0	N/A	N/A (0 branches)
app/data/models/__init__.py	0	0	0	N/A	N/A (0 branches)
app/data/models/audit_log.py	0	0	0	N/A	N/A (0 branches)
app/data/models/product.py	6	5	1	83.33%	35->36
app/data/models/purchase_order.py	12	12	0	100.00%	None (100% covered)
app/data/models/sale.py	0	0	0	N/A	N/A (0 branches)
app/data/models/stock.py	0	0	0	N/A	N/A (0 branches)
app/data/models/supplier.py	2	2	0	100.00%	None (100% covered)
app/data/models/user.py	0	0	0	N/A	N/A (0 branches)
app/presentation/__init__.py	0	0	0	N/A	N/A (0 branches)
app/presentation/authz.py	4	4	0	100.00%	None (100% covered)
app/presentation/routes/__init__.py	0	0	0	N/A	N/A (0 branches)
app/presentation/routes/auth.py	54	53	1	98.15%	145->146
app/presentation/routes/main.py	204	189	15	92.65%	59->60, 78->79, 85->86, 115->117, 122->148, 128->146, 315->321, 322->325, 326->328, 328->330, 330->332, 445->448, 454->456, 515->518, 990->992
app/presentation/routes/purchase_order.py	102	85	17	83.33%	58->59, 60->61, 92->94, 113->120, 224->226, 266->267, 268->269, 270->307, 275->278, 288->287, 294->293, 299->298, 300->301, 303->298, 335->337, 360->382, 437->439
app/presentation/routes/supplier.py	32	32	0	100.00%	None (100% covered)
scripts/backup_database.py	14	14	0	100.00%	None (100% covered)
Total / Aggregate	**694**	**635**	**59**	**91.50%**	**59 Missing Branches Across 7 Modules (9 Branching Modules at 100%)**

7. Uncovered Branch Analysis and Logic Hazards

With an overall branch coverage of **91.50%**, the system successfully exercises the overwhelming majority of conditional pathways. The 59 unreached branch transitions reside within 7 modules. A rigorous analysis reveals the exact nature of these unexercised decision paths:

- **Purchase Order Service (app/business/purchase_order.py — 20 missing branches, 85.92%):** The missing branches reflect secondary state transitions and defensive validation clauses. Transitions 27->28 and 42->43 represent null fallback guards on PO date queries. Transitions 145->146 and 167->174 guard against duplicate items. Transitions 193->194, 205->206, 210->211, and 219->220 enforce status transition rules (e.g., rejecting transition from Rejected directly to Received). Transitions 241->244, 269->270, 281->282, 286->287, 304->305, 336->337, 358->359, 372->373, 386->387, and 438->439 handle complex edge-case rollbacks when database commits fail mid-receipt.
- **Purchase Order Routes (app/presentation/routes/purchase_order.py — 17 missing branches, 83.33%):** Transitions 58->59, 60->61, 92->94, and 113->120 represent alternative query filter combinations on the order index page (e.g., filtering simultaneously by supplier ID, status enum, and custom date range). Transitions 224->226, 266->267, 268->269, 270->307, 275->278, 288->287, 294->293, 299->298, 300->301, 303->298, 335->337, 360->382, and 437->439 represent nested form validation branches when modifying dynamic line items via asynchronous requests.
- **Main Routes (app/presentation/routes/main.py — 15 missing branches, 92.65%):** Transitions 59->60, 78->79, 85->86, 115->117, 122->148, and 128->146 represent combined filter toggles on the inventory dashboard (combining text search keyword with category dropdown and in-stock toggle). Transitions 315->321, 322->325, 326->328, 328->330, 330->332, 445->448, 454->456, 515->518, and 990->992 handle fallback formatting for date picker inputs when exporting reports to CSV or PDF.
- **Stock Service (app/business/stock.py — 3 missing branches, 94.83%):** Transitions 20->21, 159->164, and 176->193 handle negative reorder threshold clamps and edge conditions when triggering low-stock notifications for items with zero safety stock.

- **Sales Service & Presentation Routes (app/business/sales.py & app/presentation/routes/auth.py):** In sales, transitions 69->70 and 123->124 handle zero-item cart validation and exact-cash change calculations. In auth, transition 145->146 handles the branch where a password reset token is structurally valid but has expired.
- **Product Model (app/data/models/product.py — 1 missing branch, 83.33%):** Transition 35->36 handles fallback string representation formatting when product price is None.

8. Conclusion and Branch Testing Boundaries

Achieving **91.50% branch coverage** demonstrates that the Electronic Store Inventory Management System's conditional logic has been rigorously exercised across both nominal and exceptional execution paths. The test suite thoroughly evaluates decision boundaries across all core business modules, ensuring that state transitions and validation rejections operate reliably.

Theoretical Boundaries of Branch Coverage: While branch coverage provides substantially stronger structural guarantees than statement coverage, it does not represent exhaustive path testing:

- **Branch Coverage vs. Path Coverage:** Branch coverage ensures that every decision point evaluates to True and False at least once, but it does not test all possible permutations of sequential branches (path coverage), which grows exponentially with cyclomatic complexity.
- **Multiple Condition Coverage (MC/DC):** In compound boolean expressions (e.g., if A and B:), branch coverage only requires that the overall condition evaluate to True and False, without proving that each sub-condition independently affects the outcome.
- **Runtime Concurrency:** Branch coverage evaluates sequential test execution and cannot verify whether race conditions or database deadlocks alter conditional outcomes under high-concurrency production deployments.